Guide — Analyser la sortie de tcpdump en ligne de commande

420-B01-HU · Surveillance préventive en cybersécurité · Oct 29, 2026 · François Paquin, CISSP

Pourquoi ce guide

tcpdump produit une ligne par paquet : sur une vraie capture, ce sont des milliers de lignes. Une dizaine de commandes Linux suffit pour en tirer des réponses, comme « qui parle le plus ? » ou « quels sites ont été visités ? ». Ce guide les présente une à une, puis montre comment les enchaîner.

Chaque exemple a été exécuté sur le fichier de pratique : la sortie affichée est celle que vous obtiendrez.

Le fichier de pratique. Il représente un petit réseau fictif (10.20.30.0/24), différent de celui du lab. Récupérez-le sur votre VM Kali :

```
cd ~/Labs && git pull
cd tcpdump/pratique
ls
```

Vous y trouverez deux fichiers :

- `pratique.pcap` : la capture elle-même, 138 paquets ;
- `pratique.txt` : la même capture déjà convertie en texte avec `tcpdump -n -r pratique.pcap > pratique.txt`.

La plupart des exemples lisent `pratique.txt` directement, pour aller plus vite. Quand un exemple a besoin d'un filtre BPF ou de l'option `-A`, il repart de `pratique.pcap`.

Méthode de travail. Tapez chaque commande vous-même plutôt que de la copier. Construisez les longues commandes un morceau à la fois, en regardant le résultat à chaque étape : c'est la méthode qui évite le plus d'erreurs.

1. Les bases : le tube et la redirection

Le tube `|` envoie la sortie d'une commande à l'entrée de la suivante. C'est ce qui permet de construire une chaîne de petits outils, chacun faisant une seule chose.

```
tcpdump -n -r pratique.pcap | wc -l
```

```
138
```

tcpdump lit la capture et écrit 138 lignes ; au lieu de les afficher, le tube les donne à `wc -l`, qui les compte.

Rediriger vers un fichier

Symbole	Effet
`> fichier`	Écrit la sortie dans le fichier (l'écrase s'il existe)
`>> fichier`	Ajoute la sortie à la fin du fichier

Symbole	Effet
`2> /dev/null`	Jette les messages d'erreur

```
tcpdump -n -r pratique.pcap > pratique.txt
```

Attention : `>` écrase le fichier sans demander de confirmation.

Le message « reading from file »

tcpdump affiche toujours `reading from file pratique.pcap, link-type EN10MB (Ethernet)...` en tête. Ce message passe par la sortie d'erreur : il n'entre pas dans le tube et n'est pas compté par `wc -l`. Pour le faire disparaître, ajoutez `2> /dev/null`.

De même, `tcpdump ... | head` peut afficher `Unable to write output: Broken pipe` : `head` a fermé le tube après avoir reçu assez de lignes. Ce message est sans conséquence.

Regarder un fichier

Commande	Effet
`cat fichier`	Affiche tout le fichier
`less fichier`	Affiche page par page (`q` pour quitter, `/mot` pour chercher)
`head -3 fichier`	Les 3 premières lignes (10 par défaut)
`tail -2 fichier`	Les 2 dernières lignes
`wc -l fichier`	Le nombre de lignes

```
head -3 pratique.txt
```

```
15:06:40.000000 IP 10.20.30.11.56771 > 10.20.30.2.53: 62348+ A? meteo.gc.ca. (29)
15:06:40.010000 IP 10.20.30.2.53 > 10.20.30.11.56771: 62348 1/0/0 A 10.99.0.30 (56)
15:06:40.050000 IP 10.20.30.11.43363 > 10.99.0.30.80: Flags [S], seq 958999843, win
502, length 0
```

Conseil. Avant de construire une commande longue, regardez toujours quelques lignes avec `head` pour savoir à quoi ressemblent les données.

2. grep : garder les lignes qui contiennent un motif

`grep motif fichier` affiche seulement les lignes qui contiennent le motif. Mettez le motif entre apostrophes dès qu'il contient un espace ou un caractère spécial.

```
grep 'HTTP: GET' pratique.txt | head -3
```

```
15:06:40.062000 IP 10.20.30.11.43363 > 10.99.0.30.80: Flags [P.], seq 1:107, ack 1,
win 502, length 106: HTTP: GET /vieux-lien HTTP/1.1
15:06:42.628747 IP 10.20.30.14.43628 > 10.99.0.40.80: Flags [P.], seq 1:113, ack 1,
win 502, length 112: HTTP: GET /index.html HTTP/1.1
15:06:44.528056 IP 10.20.30.14.51392 > 10.99.0.30.80: Flags [P.], seq 1:107, ack 1,
win 502, length 106: HTTP: GET /index.html HTTP/1.1
```

Les options à connaître

Option	Effet	Exemple	Résultat
`-c`	Compte les lignes au lieu de les afficher	`grep -c 'A?' pratique.txt`	`15` requêtes DNS
`-v`	Inverse : garde les lignes qui ne contiennent pas le motif	`grep -v 'Flags' pratique.txt`	tout sauf TCP (DNS, ICMP)
`-i`	Ignore majuscules et minuscules	`grep -i 'not found' pratique.txt`	trouve `Not Found`
`-n`	Ajoute le numéro de ligne devant	`grep -n '403' pratique.txt`	`47:15:06:51...`
`-o`	N'affiche que la partie qui correspond	`grep -o 'GET [^ ]*' pratique.txt`	`GET /vieux-lien`
`-E`	Active les expressions étendues (alternatives, `+`, `?`)	voir plus bas	
`-A 2` / `-B 4`	Affiche aussi 2 lignes après / 4 lignes avant	utile avec `tcpdump -A`	

L'option `-o` est la plus utile en analyse : elle transforme une longue ligne en une seule information.

```
grep -o 'GET [^ ]*' pratique.txt | head -4
```

```
GET /vieux-lien
GET /index.html
GET /index.html
GET /
```

Mini-guide des expressions régulières

Un motif `grep` est une expression régulière : certains caractères ont un sens spécial.

Symbole	Sens	Exemple	Trouve
`.`	N'importe quel caractère	`10.20`	`10.20`, mais aussi `10X20`
`\.`	Un vrai point	`www\.`	`www.` seulement
`[abc]`	Un caractère parmi ceux-là	`[0-9]`	un chiffre
`[^ ]`	Tout caractère sauf l'espace	`[^ ]*`	un « mot » entier
`*`	Le caractère précédent, 0 fois ou plus	`[0-9]*`	`''`, `4`, `443`
`+`	1 fois ou plus (avec `-E`)	`[0-9]+`	`4`, `443`
`^`	Début de ligne	`^15:07`	lignes de 15 h 07

Symbole	Sens	Exemple	Trouve
`$`	Fin de ligne	`OK$`	lignes finissant par OK
`{3}`	Exactement 3 fois (avec `-E`)	`[0-9]{3}`	`404`

Alternatives. Avec `-E`, la barre verticale veut dire « ou », et les parenthèses regroupent :

```
grep -E ' (403|404) ' pratique.txt | wc -l
```

```
7
```

Piège : un motif trop large

Sans les espaces autour, le même motif trouve 9 lignes au lieu de 7. Voici les 2 lignes de trop :

```
grep -E '403|404' pratique.txt | grep -vE ' (403|404) '
```

```
15:07:01.640378 IP 10.99.0.40.80 > 10.20.30.14.49504: Flags [S.], seq 1965615738, ack
1360031913, win 502, length 0
15:07:11.404876 IP 10.20.30.14.44444 > 10.20.30.5.80: Flags [S], seq 2113076313, win
1024, length 0
```

`403` se cache dans le numéro d'accusé de réception `1360031913`, et `404` dans l'heure `15:07:11.404876`. `grep` cherche des caractères, pas des nombres : rendez le motif précis (espaces, contexte comme `HTTP/1.1 404`), puis vérifiez toujours quelques lignes du résultat.

3. cut et tr : découper et transformer

cut : garder certains morceaux d'une ligne

`cut` découpe chaque ligne selon un séparateur (`-d`) et garde les champs demandés (`-f`).

Commande	Résultat
`echo 10.20.30.11.56771 | cut -d. -f1-4`	`10.20.30.11` (l'adresse sans le port)
`echo 10.20.30.11.56771 | cut -d. -f5`	`56771` (le port seul)
`head -1 pratique.txt | cut -c1-8`	`15:06:40` (les caractères 1 à 8 : l'heure à la seconde)

- `-d.` : le séparateur est le point. Pour l'espace, écrivez `-d' '`.
- `-f1-4` : champs 1 à 4 ; `-f5` : champ 5 seul ; `-f1,3` : champs 1 et 3.
- `-c1-8` : coupe par position de caractère plutôt que par champ.

Pourquoi c'est si utile avec tcpdump. tcpdump colle le port à l'adresse IP avec un point : `10.20.30.11.56771`. Pour compter des adresses, il faut d'abord retirer le port, sinon chaque connexion compte à part. `cut -d. -f1-4` le fait en une seule étape.

Limite. `cut -d' '` ne gère pas bien plusieurs espaces de suite : chaque espace compte comme un séparateur. Pour découper en colonnes séparées par des espaces, préférez `awk` (section 4).

tr : remplacer ou supprimer des caractères

`tr` travaille caractère par caractère. Il lit seulement l'entrée standard : on l'utilise toujours après un tube.

Commande	Résultat	Usage
`echo meteo.gc.ca | tr a-z A-Z`	`METEO.GC.CA`	Passer en majuscules
`echo 10.20.30.5.22 | tr '.' ' '`	`10 20 30 5 22`	Remplacer un caractère par un autre
`tr -d '\n'`	colle toutes les lignes en une	Reconstituer une chaîne découpée
`echo 'un  deux' | tr -s ' '`	`un deux`	Fusionner les espaces répétés

`tr -d '\n'` sert dans le lab pour recoller des morceaux de texte répartis sur plusieurs lignes. Exemple :

```
printf 'a\nb\nc\n' | tr -d '\n'
```

```
abc
```

4. awk : travailler par colonnes

`awk` découpe chaque ligne en colonnes séparées par des espaces et les numérote `$1`, `$2`, `$3`... C'est l'outil le plus puissant du guide ; on en utilise surtout trois usages.

Les colonnes d'une ligne tcpdump

Avec `-n`, une requête DNS se découpe ainsi :

```
15:06:40.000000 IP 10.20.30.11.56771 > 10.20.30.2.53: 62348+ A? meteo.gc.ca. (29)
```

Colonne	Contenu	Valeur ici
`$1`	Heure	`15:06:40.000000`
`$2`	Protocole	`IP`
`$3`	Source (IP + port)	`10.20.30.11.56771`
`$4`	Flèche	`>`
`$5`	Destination (IP + port + deux-points)	`10.20.30.2.53:`
`$6`	Identifiant DNS	`62348+`
`$7`	Type demandé	`A?`
`$8`	Nom demandé	`meteo.gc.ca.`
`$NF`	Dernière colonne	`(29)`

`$1` à `$5` sont les mêmes pour tous les paquets IP. À partir de `$6`, tout dépend du protocole : pour TCP, `$6` vaut `Flags` et `$7` les drapeaux. Vérifiez toujours avec `head`.

Usage 1 : afficher des colonnes

```
head -2 pratique.txt | awk '{print $1, $3, $5}'
```

```
15:06:40.000000 10.20.30.11.56771 10.20.30.2.53:
15:06:40.010000 10.20.30.2.53 10.20.30.11.56771:
```

- Les apostrophes sont obligatoires : sans elles, le shell remplacerait `$3` par une variable vide.
- La virgule dans `print` ajoute un espace entre les colonnes.
- `NF` est le nombre de colonnes de la ligne : `$NF` est la dernière, `$(NF-1)` l'avant-dernière.

```
awk '/HTTP: GET/ {print $(NF-1)}' pratique.txt | head -3
```

```
/vieux-lien
/index.html
/index.html
```

Ici, la page demandée est toujours l'avant-dernière colonne, juste avant `HTTP/1.1`.

Usage 2 : filtrer sur une condition

Une condition placée avant les accolades choisit les lignes traitées.

Commande	Garde les lignes où...
`awk '/HTTP: GET/' f`	la ligne contient `HTTP: GET` (comme `grep`)
`awk '$7 == "A?" {print $8}' f`	la colonne 7 vaut exactement `A?`
`awk '$3 == "10.20.30.5.22"' f`	la source est exactement ce couple IP + port
`awk '$3 ~ /^10\.99\./' f`	la colonne 3 commence par `10.99.`

```
awk '$7 == "A?" {print $8}' pratique.txt | head -3
```

```
meteo.gc.ca.
www.wikipedia.org.
meteo.gc.ca.
```

Avantage sur `grep` : la condition porte sur une colonne précise, ce qui évite le piège de la section 2 (un nombre trouvé au mauvais endroit).

Usage 3 : compter et additionner

Le bloc `END` s'exécute une seule fois, après la dernière ligne.

```
grep -o 'length [0-9]*' pratique.txt | awk '{s += $2} END {print s}'
```

```
2223
```

`grep -o` extrait `length 106`, `length 0`... ; `awk` additionne la colonne 2 dans la variable `s`, puis l'affiche à la fin. Résultat : 2 223 octets de données dans toute la capture.

`awk 'END {print NR}' fichier` affiche le nombre de lignes (`NR` = numéro de la ligne courante), comme `wc -l`.

Changer de séparateur avec -F

```
echo 10.20.30.11.56771 | awk -F. '{print $5}'
```

```
56771
```

`-F.` découpe selon le point au lieu de l'espace, comme `cut -d.`.

5. sed : modifier le texte au passage

`sed` lit chaque ligne, lui applique une modification et la réécrit. Le fichier d'origine n'est pas modifié : seule la sortie change.

Remplacer : s/ancien/nouveau/

Commande	Résultat
`echo 'a.b.c' | sed 's/\./-/'`	`a-b.c` : seulement la 1re occurrence
`echo 'a.b.c' | sed 's/\./-/g'`	`a-b-c` : `g` = toutes les occurrences
`head -1 pratique.txt | sed 's/IP/IPv4/'`	`15:06:40.000000 IPv4 10.20.30.11...`

Le motif à chercher est une expression régulière, comme pour `grep` : un point s'écrit `\.`.

Retirer quelque chose : remplacer par rien

La destination tcpdump (`$5`) finit par un deux-points gênant :

```
head -1 pratique.txt | awk '{print $5}' | sed 's/:$//'
```

```
10.20.30.2.53
```

`:$` = un deux-points en fin de ligne ; la partie « nouveau » est vide, donc il disparaît.

De la même façon, `sed 's/\.[0-9]*$//'` retire le dernier point et ce qui suit :

```
echo 10.20.30.11.56771 | sed 's/\.[0-9]*$//'
```

```
10.20.30.11
```

Piège. Sur une ligne ICMP, il n'y a pas de port, et la même commande retire le dernier octet de l'adresse :

```
echo 10.20.30.12 | sed 's/\.[0-9]*$//'
```

```
10.20.30
```

Pour retirer le port, `cut -d. -f1-4` est plus sûr : il garde toujours exactement quatre octets.

Afficher ou supprimer des lignes

Commande	Effet
`sed -n '3,4p' pratique.txt`	Affiche seulement les lignes 3 et 4 (`-n` = rien par défaut, `p` = afficher)
`sed '/Flags/d' pratique.txt`	Supprime les lignes qui contiennent `Flags` (il en reste 36 : DNS et ICMP)

Extraire avec un groupe de capture

Avec `-E`, les parenthèses capturent une partie du motif ; `\1` la réutilise dans le remplacement.

```
grep 'HTTP: GET' pratique.txt | sed -E 's/.*GET ([^ ]*).*/\1/' | head -3
```

```
/vieux-lien
/index.html
/index.html
```

Lecture du motif : `.*` tout ce qui précède, `GET ` , puis `([^ ]*)` le mot suivant (capturé), puis `.*` tout le reste. La ligne entière est remplacée par le mot capturé seulement.

Ce groupe de capture est la technique la plus générale pour isoler une information au milieu d'une ligne. Dans les cas simples, `grep -o` ou `awk` sont plus lisibles.

6. sort et uniq : trier et compter

Ces deux commandes vont presque toujours ensemble : `sort` regroupe les lignes identiques, `uniq` les fusionne.

sort : trier

Option	Effet	`10`, `9`, `100` deviennent
(aucune)	Tri alphabétique, caractère par caractère	`10`, `100`, `9`
`-n`	Tri numérique	`9`, `10`, `100`
`-rn`	Numérique, du plus grand au plus petit	`100`, `10`, `9`
`-u`	Trie et retire les doublons	
`-t. -k4 -n`	Séparateur point, tri numérique sur le 4e champ	(adresses IP, voir plus bas)

Le tri alphabétique place `100` avant `9`, parce que le caractère `1` vient avant `9`. Pour des nombres, utilisez toujours `-n`.

Même problème avec les adresses IP :

```
printf '10.20.30.14\n10.20.30.5\n10.20.30.11\n' | sort -t. -k4 -n
```

```
10.20.30.5
10.20.30.11
10.20.30.14
```

Sans `-t. -k4 -n`, l'adresse `.5` arriverait en dernier.

uniq : fusionner les doublons consécutifs

Option	Effet
(aucune)	Ne garde qu'un exemplaire de chaque ligne répétée
`-c`	Ajoute devant chaque ligne le nombre de répétitions
`-d`	N'affiche que les lignes en double

Piège : uniq sans sort

`uniq` ne compare une ligne qu'à la précédente. Sans tri, les doublons séparés ne sont pas fusionnés :

```
printf 'a\nb\na\na\n' | uniq -c
```

```
      1 a
      1 b
      2 a
```

Avec `sort` d'abord, le compte est juste :

```
printf 'a\nb\na\na\n' | sort | uniq -c
```

```
      3 a
      1 b
```

Le trio sort | uniq -c | sort -rn

C'est la combinaison la plus utilisée en analyse de journaux. Elle répond à « qu'est-ce qui revient le plus souvent ? ».

```
grep -o 'A? [^ ]*' pratique.txt | sort | uniq -c | sort -rn
```

```
      6 A? www.wikipedia.org.
      4 A? www.cegepoutaouais.qc.ca.
      3 A? meteo.gc.ca.
      1 A? www.ville.gatineau.qc.ca.
      1 A? depot.exemple.net.
```

1. `sort` regroupe les lignes identiques.
2. `uniq -c` les compte.
3. `sort -rn` classe les comptes du plus grand au plus petit.

Ajoutez `| head -5` pour ne garder que le palmarès. Le bas de la liste est souvent plus intéressant en sécurité : ce qui n'apparaît qu'une fois, comme `depot.exemple.net`, mérite un coup d'œil.

7. Recettes : combiner les commandes

Une longue commande se construit toujours de la même façon : on ajoute un morceau, on regarde le résultat, on ajoute le suivant. Chaque recette ci-dessous montre ces étapes.

Recette 1 : qui émet le plus de paquets ?

Étape 1 : extraire la source (colonne 3).

```
awk '{print $3}' pratique.txt | head -3
```

```
10.20.30.11.56771
10.20.30.2.53
10.20.30.11.43363
```

Étape 2 : retirer le port.

```
awk '{print $3}' pratique.txt | cut -d. -f1-4 | head -3
```

```
10.20.30.11
10.20.30.2
10.20.30.11
```

Étape 3 : compter et classer (on retire le `head`).

```
awk '{print $3}' pratique.txt | cut -d. -f1-4 | sort | uniq -c | sort -rn
```

```
     41 10.20.30.14
     20 10.20.30.13
     20 10.20.30.11
     16 10.20.30.2
     12 10.99.0.40
      8 10.99.0.10
      7 10.20.30.5
      6 10.99.0.30
      3 10.20.30.12
      2 10.99.0.50
      2 10.99.0.20
      1 10.20.30.1
```

10.20.30.14 émet deux fois plus que les autres postes : c'est le premier à examiner.

Recette 2 : quels domaines ont été demandés au DNS ?

On combine un filtre BPF (dans tcpdump) et le trio de comptage (dans le shell).

```
tcpdump -n -r pratique.pcap 'udp dst port 53' | awk '{print $8}' | sort | uniq -c |
sort -rn
```

```
      6 www.wikipedia.org.
      4 www.cegepoutaouais.qc.ca.
      3 meteo.gc.ca.
      1 www.ville.gatineau.qc.ca.
      1 depot.exemple.net.
```

- `'udp dst port 53'` : seulement les questions envoyées au serveur DNS. Sans `dst`, les réponses s'ajoutent, et leur colonne 8 contient autre chose.
- `awk '{print $8}'` : le nom demandé.

Recette 3 : quels codes de réponse HTTP ?

```
grep -o 'HTTP/1.1 [0-9][0-9][0-9]' pratique.txt | sort | uniq -c | sort -rn
```

```
      8 HTTP/1.1 200
      4 HTTP/1.1 404
      3 HTTP/1.1 403
```

`[0-9][0-9][0-9]` = exactement trois chiffres. Trois refus `403` sur `/admin` méritent qu'on regarde qui les a provoqués.

Recette 4 : quelles pages ont été demandées ?

```
grep -o 'GET [^ ]*' pratique.txt | sort | uniq -c | sort -rn
```

```
      4 GET /vieux-lien
      3 GET /contact
      3 GET /admin
      2 GET /index.html
      2 GET /
      1 GET /outil.sh
```

Recette 5 : quels ports ont répondu « ouvert » ?

Un port ouvert répond à un SYN par un SYN-ACK. On filtre ces réponses dans tcpdump, puis on extrait le port source.

```
tcpdump -n -r pratique.pcap 'src host 10.20.30.5 and tcp[tcpflags] == tcp-syn|tcp-
ack' | awk '{print $3}' | cut -d. -f5 | sort -n
```

```
22
80
```

- `cut -d. -f5` : cinquième champ séparé par les points, c'est-à-dire le port.
- Le filtre BPF est entre apostrophes : sinon le shell prendrait la barre verticale de `tcp-syn|tcp-ack` pour un tube.

Recette 6 : repérer une rafale de paquets

Les 8 premiers caractères d'une ligne sont l'heure à la seconde. Les compter montre le rythme du trafic.

```
cut -c1-8 pratique.txt | uniq -c | sort -rn | head -3
```

```
     16 15:07:11
      8 15:07:12
      8 15:07:10
```

15:07:11 compte deux fois plus de paquets que les autres secondes. `grep '^15:07:11' pratique.txt` montre pourquoi : un balayage de ports et des ping.

Ici, `uniq -c` sans `sort` fonctionne, parce que les lignes sont déjà dans l'ordre chronologique : les secondes identiques se suivent.

Recette 7 : quels logiciels ont navigué ?

Le User-Agent est dans le contenu du paquet : il faut l'option `-A` de tcpdump, donc repartir du fichier `.pcap`.

```
tcpdump -n -A -r pratique.pcap 'tcp port 80' | grep 'User-Agent' | sort | uniq -c | sort -rn
```

```
     14 User-Agent: Mozilla/5.0 (X11; Linux x86_64) Firefox/131.0
      1 User-Agent: curl/8.5.0
```

Un seul téléchargement fait avec `curl`, un outil en ligne de commande : l'exercice 6 vous demande de retrouver lequel.

Recette 8 : voir ce qui entoure une ligne

`grep -A` et `grep -B` affichent les lignes après et avant chaque correspondance. Avec `tcpdump -A`, c'est la façon de lire une requête HTTP complète autour d'un mot-clé.

```
tcpdump -n -A -r pratique.pcap 'tcp port 80' | grep -A3 'GET /contact' | grep -E 'Host:|User-Agent:' | sort | uniq -c
```

```
      1 Host: www.cegepoutaouais.qc.ca
      2 Host: www.wikipedia.org
      3 User-Agent: Mozilla/5.0 (X11; Linux x86_64) Firefox/131.0
```

La page `/contact` a été demandée 3 fois, sur deux sites, toujours par Firefox.

- `grep -A3` garde chaque ligne trouvée et les 3 suivantes. On prend 3 lignes plutôt que 2 parce que l'en-tête binaire affiché par `-A` contient parfois des retours à la ligne : les lignes `Host:` et `User-Agent:` ne sont pas toujours juste après.
- Le second `grep` ne garde que les lignes utiles parmi ce contexte.

Regardez toujours la sortie brute de `grep -A3` avant d'ajouter le filtre : c'est elle qui montre combien de lignes de contexte sont nécessaires.

8. Exercices

Répondez à chaque question avec une seule commande, construite étape par étape. Notez la commande finale : elle compte autant que la réponse.

1. Combien d'hôtes ont répondu au ping ? Donnez leurs adresses, triées dans l'ordre numérique.
2. Quel poste fait le plus de requêtes DNS, et combien ?
3. Donnez la liste des domaines demandés, sans doublon, sans le point final, en ordre alphabétique.
4. Quels postes ont reçu une erreur HTTP 403 ou 404, et combien chacun ?
5. Quels ports 10.20.30.14 a-t-il testés sur 10.20.30.5 ? Lesquels sont ouverts ?
6. Un fichier a été téléchargé avec `curl`. Quel poste l'a fait, quel fichier et sur quel site ?

7. Combien de requêtes HTTP `GET` chaque poste a-t-il envoyées ?

Solutions

Essayez d'abord. Il existe souvent plusieurs bonnes commandes : celles-ci n'en sont qu'une.

1. 3 hôtes : 10.20.30.1, 10.20.30.2 et 10.20.30.5.

```
grep 'ICMP echo reply' pratique.txt | awk '{print $3}' | sort -t. -k4 -n
```

2. 10.20.30.14, avec 7 requêtes.

```
grep 'A? ' pratique.txt | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -rn
```

3. depot.exemple.net, meteo.gc.ca, www.cegepoutaouais.qc.ca, www.ville.gatineau.qc.ca, www.wikipedia.org.

```
grep -o 'A? [^ ]*' pratique.txt | awk '{print $2}' | sed 's/\.$//' | sort -u
```

4. 10.20.30.11 (3), 10.20.30.14 (3), 10.20.30.13 (1). La destination de la réponse est le poste qui a fait la demande.

```
grep -E 'HTTP/1.1 (403|404)' pratique.txt | awk '{print $5}' | sed 's/:$//' | cut -d. -f1-4 | sort | uniq -c
```

5. Ports testés : 21, 22, 23, 80, 443 et 3389. Ouverts : 22 et 80.

```
tcpdump -n -r pratique.pcap 'src host 10.20.30.14 and dst host 10.20.30.5' | awk '{print $5}' | sed 's/:$//' | cut -d. -f5 | sort -n
tcpdump -n -r pratique.pcap 'src host 10.20.30.5 and tcp[tcpflags] == tcp-syn|tcp-ack' | awk '{print $3}' | cut -d. -f5 | sort -n
```

6. 10.20.30.13 a téléchargé `/outil.sh` sur depot.exemple.net.

```
tcpdump -n -A -r pratique.pcap 'tcp port 80' | grep -A3 'GET /outil' | grep -E 'GET|Host:|User-Agent:'
```

Recette 7 pour repérer `curl`, puis la recette 8 pour lire la requête. La première ligne affichée donne la source : 10.20.30.13.

7. 10.20.30.14 : 7 ; 10.20.30.13 : 4 ; 10.20.30.11 : 4.

```
grep 'HTTP: GET' pratique.txt | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -rn
```

9. Aide-mémoire

Je veux...	Commande
Compter les lignes	`wc -l`
Voir le début / la fin	`head -5` / `tail -5`
Garder les lignes qui contiennent X	`grep 'X'`
Garder les lignes qui ne contiennent pas X	`grep -v 'X'`
Compter les lignes qui contiennent X	`grep -c 'X'`

Je veux...	Commande
Extraire seulement le morceau trouvé	`grep -o 'GET [^ ]*'`
Chercher X ou Y	`grep -E 'X|Y'`
Voir les lignes autour	`grep -A3 'X'` (après), `grep -B3 'X'` (avant)
Afficher la colonne 3	`awk '{print $3}'`
Afficher la dernière colonne	`awk '{print $NF}'`
Garder les lignes où la colonne 7 vaut A?	`awk '$7 == "A?"'`
Additionner la colonne 2	`awk '{s += $2} END {print s}'`
Retirer le port d'une adresse	`cut -d. -f1-4`
Garder seulement le port	`cut -d. -f5`
Garder l'heure à la seconde	`cut -c1-8`
Retirer un caractère en fin de ligne	`sed 's/:$//'`
Remplacer partout	`sed 's/ancien/nouveau/g'`
Supprimer les lignes qui contiennent X	`sed '/X/d'`
Passer en majuscules	`tr a-z A-Z`
Coller toutes les lignes	`tr -d '\n'`
Trier des nombres	`sort -n` (`-rn` décroissant)
Trier des adresses IP	`sort -t. -k4 -n`
Liste sans doublon	`sort -u`
Compter les occurrences	`sort | uniq -c | sort -rn`
Jeter les messages d'erreur	`2> /dev/null`
Enregistrer le résultat	`> fichier.txt`

Les trois règles d'or

1. Regardez quelques lignes avec `head` avant de choisir une colonne ou un motif.
2. Construisez la commande un morceau à la fois, en vérifiant chaque étape.
3. Mettez les motifs, les programmes `awk` et les filtres tcpdump entre apostrophes.